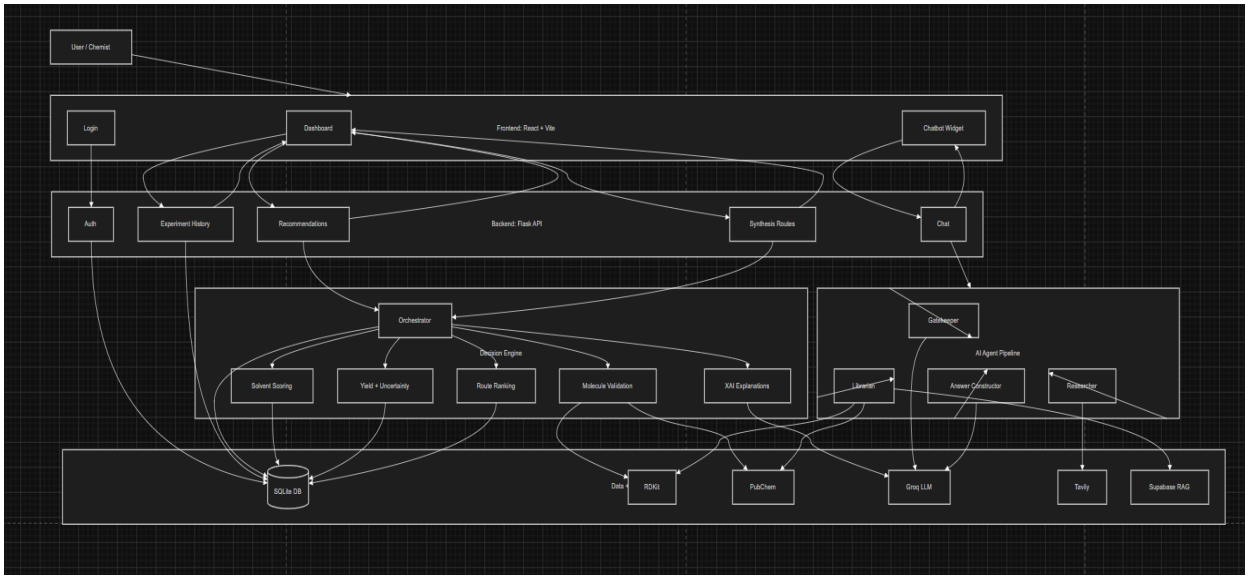

Description:
GreenRoute AI is built as a full-stack Flask and React system for green chemistry decision support. The architecture separates the user interface, backend APIs, decision engine, AI chatbot pipeline, and external data sources so each part has a clear responsibility.

The Frontend, built with React and Vite, is the main interface used by the chemist. It contains the login screen, the GreenRoute dashboard, and the floating chatbot widget. Through the dashboard, the user can enter a target molecule by SMILES or common name, adjust optimization weights, request greener solvent recommendations, compare synthesis routes, approve a result, and view experiment history. The chatbot widget allows the user to ask chemistry-related questions without leaving the platform.

The Backend, implemented with Flask, exposes protected REST API endpoints. Authentication is handled through the Auth API,

while recommendation requests are sent to the Recommendation API. Synthesis route comparisons are handled by the Synthesis Routes API, approved experiments are stored through the Experiment History API, and chatbot messages are sent through the Chat API. These endpoints use bearer tokens so only authenticated users can access the main platform functions.

The Decision Engine is coordinated by the Orchestrator. It receives molecule and configuration data from the backend and manages the core green chemistry workflow. It validates molecules using RDKit and PubChem, ranks solvent alternatives using green chemistry metrics, estimates yield and uncertainty, ranks synthesis routes using atom economy, E-factor, and reaction step count, and generates explainable AI text for the final recommendations. Results are returned to the frontend as ranked cards with metrics, explanations, warnings, and source traceability.

The AI Agent Pipeline powers the floating chatbot. It is composed of four agent roles. The Gatekeeper checks whether the user's question is relevant to chemistry, chemoinformatics, toxicity, molecular properties, or biology. The Librarian retrieves scientific and molecular information using RDKit, PubChem, and optional Supabase RAG context. The Researcher can perform external web search through Tavily when configured. The Answer Constructor combines the available evidence and uses Groq to generate a structured response for the user.

The Data and External Services layer supports both the decision engine and chatbot. SQLite stores users, authentication tokens,

solvent data, synthesis routes, saved sessions, and experiment history. RDKit is used for molecule parsing and molecular property calculations. PubChem provides external compound and assay information. Groq is used for AI-generated explanations and chatbot response construction. Tavily and Supabase RAG are optional services that enrich chatbot answers with web search and document retrieval.

Overall, the system follows a human-in-the-loop design. The platform can recommend greener solvents and synthesis routes, but the chemist remains responsible for reviewing and approving the final decision before it is logged as a validated experiment.